Perceiver & Perceiver IO

Implementation and Experimental Analysis

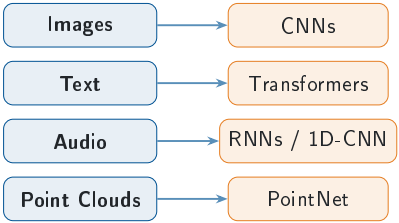
Francesco Gigli and Corso Vignoli

Università degli Studi di Firenze
Course B031278 — Deep Learning

Agenda

The Problem: Modality-Specific Architectures

One modality, one architecture



Each data type is usually paired with a dedicated model and domain-specific assumptions about structure.

The scaling bottleneck

Self-attention cost

$$\mathcal{O}(M^2 \cdot d)$$

grows quadratically with input length M

Input representation	M
16×16 image patches	256
raw 224×224 pixels	50,176

Raw pixels make standard attention impractical.

Core question: can one architecture handle arbitrary inputs without quadratic cost?

The Perceiver Solution

Core idea: learned latents read the input once; expensive processing stays in latent space.

1. Read

Cross-attention

M inputs $\rightarrow$ $N \ll M$ latents

2. Process

Latent block

transform N latent slots

3. Reuse

Shared weights

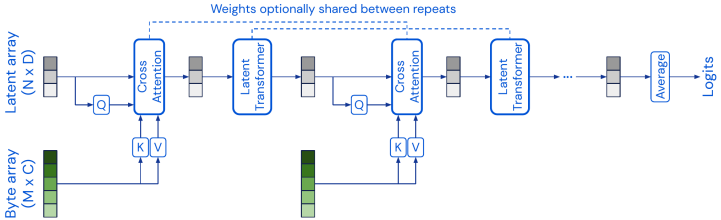
repeat the same processor

Complexity shift

Read the full input once; repeat the expensive work in the compact latent space $N \ll M$.

Operation	Cost
Read input once	$\mathcal{O}(M \cdot N)$
Process latents for L layers	$\mathcal{O}(L \cdot N^2)$
Perceiver	$\mathcal{O}(MN + LN^2)$

Perceiver Architecture



Encode and process

- $X \in \mathbb{R}^{M \times C}$ supplies keys and values.
- $Z \in \mathbb{R}^{N \times D}$ supplies queries, with $N \ll M$.
- Cross-attention maps $X \to Z$: $\mathcal{O}(MN)$.

Latent core and output

- Latent Transformer updates Z only.
- Repeated blocks may share weights.
- Original Perceiver pools Z; Perceiver IO uses output queries.

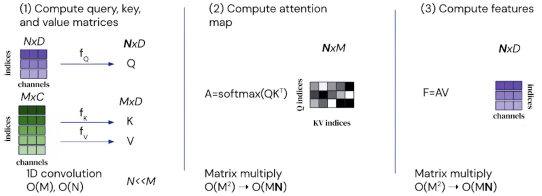
Cross-Attention: The Key Mechanism

Formally:

$$Q = X_{\text{latent}}\, W_Q \in \mathbb{R}^{N \times d_k}$$
$$K = X_{\text{input}}\, W_K \in \mathbb{R}^{M \times d_k}$$
$$V = X_{\text{input}}\, W_V \in \mathbb{R}^{M \times d_v}$$

$$\text{CrossAttn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

The latents are the queries, the input provides keys and values $\Rightarrow$ informational *bottleneck*.

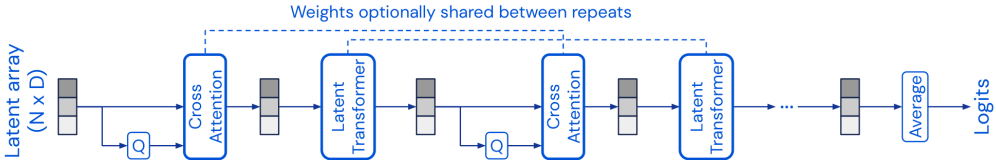


Key insight:

- Attention matrix: $N \times M$ (not $M \times M$)
- Complexity: $\mathcal{O}(N \cdot M \cdot d_k)$
- Decouples depth from input size

Latent Transformer & Weight Sharing

Once information is in the latent array, depth is spent by reusing the same processor on a compact state.



Weight sharing

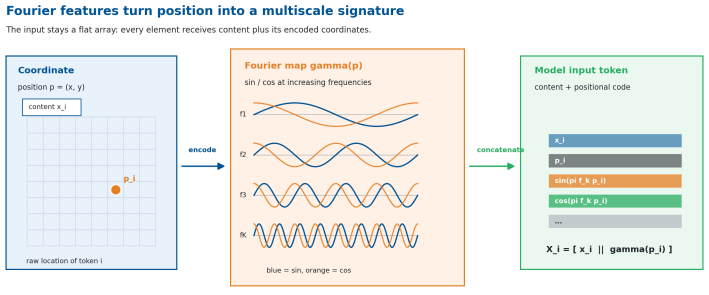
- The same Latent Transformer block B_θ is reused at every repeat, with shared parameters θ.
- Unrolling depth behaves like an RNN over the latent state: $Z_{t+1} = B_\theta(Z_t)$.

Why it matters

- Iterative refinement without instantiating a new processor at each step.
- Parameter count stays fixed as depth grows; computation stays in the N-slot latent space.

Fourier Positional Encoding

Perceiver receives a flat array, so location has to be part of each input token.



General recipe

Use domain coordinates: images (u, v), point clouds (x, y, z), or text index t; sine/cosine bands encode them at multiple scales.

Input seen by attention

The array is still flat, but each token is enriched before projection: $X_i = [\, x_i \,\|\, \gamma(p_i) \,]$.

Perceiver Forward Pass: Shape Checkpoints

Data flow in the original Perceiver:

1. Input array: each token stacks content x with a Fourier position code $\gamma(\mathrm{p})$,

$$X = [\,\mathrm{x} \,\|\, \gamma(\mathrm{p})\,] \in \mathbb{R}^{M \times C}.$$

2. Learned latent array:

$$Z_0 \in \mathbb{R}^{N \times D}, \qquad N \ll M$$

3. Cross-attention reads input:

$$Q = ZW_Q, \quad K = XW_K, \quad V = XW_V$$

4. Latent Transformer applies self-attention + FFN only on N latents.

Shape checkpoints:

Object	Shape	Meaning
X	$M \times C$	input array (content+pos)
Z_0	$N \times D$	learned latents
Q	$N \times d_k$	latent queries
K, V	$M \times d_k, d_v$	input keys/values
A	$N \times M$	attention map

Core idea

The input size M affects only the cross-attention read; depth is spent in the compact latent space.

Original Perceiver Output: Pooling Classifier

Fixed-shape output head:

$$Z_T = \mathrm{Encoder}(X, Z_0)$$

$$z = \frac{1}{N}\sum_{i=1}^{N} Z_T[i]$$

$$\mathrm{logits} = zW_{\mathrm{cls}} + b_{\mathrm{cls}}$$

- Mean pooling collapses all latents to one vector
- The classifier predicts one global label
- This is ideal for classification, not dense structured outputs

Where FFN and sharing fit:

- Each latent block is pre-norm attention, residual, FFN, residual
- FFN adds non-linearity after attention mixing
- Weight sharing repeats the processor over iterations

Fixed-shape output

Pooling collapses the N latents into a single vector, so this head emits one global prediction per input.

Computational Complexity Analysis

Model	Self-Attn	Cross-Attn	Total
Standard Transformer	$\mathcal{O}(M^2 d)$	—	$\mathcal{O}(M^2 d \cdot L)$
Perceiver	$\mathcal{O}(N^2 d)$	$\mathcal{O}(MNd)$	$\mathcal{O}((MN + N^2 L)\, d)$

Symbols:

- M: number of input elements
- N: number of latent slots, with $N \ll M$
- L: number of latent processing layers
- d: hidden channel dimension

Interpretation:

- The full input is read through cross-attention
- Repeated self-attention happens only in latent space
- If N is fixed, depth becomes independent of input length
- Concrete sizes are evaluated in the experiments

GELU Activation & LAMB Optimizer

GELU (Gaussian Error Linear Unit):

$$\mathrm{GELU}(x) = x \cdot \Phi(x) \approx x \cdot \sigma(1.702\,x)$$

- Smooth activation used in Transformer architectures
- Allows small negative gradient flow

LAMB (Layer-wise Adaptive Moments):

$$r_t^{(l)} = \frac{\|w_t^{(l)}\|}{\|w_t^{(l)} + \eta\,\hat{u}_t^{(l)}\|}$$

Per-layer trust ratio rescales updates and stabilizes large-batch training.

Why LAMB for Perceiver?

- Cross-attention bottleneck creates high variance gradients
- Standard Adam can destabilize with large batches
- LAMB safely scales batch sizes up to 1024

Optimizer	Large Batches	Layer-wise LR
Adam	Diverges	Global
LAMB	**Stable**	**Per-layer trust**

Implementation

Aspect	Paper implementation	Project code
Core model	Cross-attention into learned latents, latent self-attention, FFN, weight sharing	Same modules; weight sharing is exposed as an ablation switch
Model scale	Large vision/language runs: up to 512×1024 latents and 201M-param IO	Smaller runs: CIFAR-10 96×384, ModelNet40/text 128×512
Data setting	ImageNet, ModelNet40, large language pretraining corpora	CIFAR-10, ModelNet40, WikiText-103 byte-level MLM, GLUE fine-tuning
Training budget	Distributed accelerators, batches up to 512–1024	Single-GPU budget, batches 32–128, LAMB retained for stability
Practical additions	Minimal regularization in the reference setup	Dropout, logging, attention-map extraction, and reproducible ablations

The implementation keeps the architectural mechanism intact; the main differences are scale, data budget and training resources.

Implementation Differences from the Paper

Choice	Original paper (ImageNet)	This project
Input tokenization	Raw pixels, no patching ($M = 50{,}176$)	**Raw pixels, no patching** ($M = 1{,}024$ on 32×32)
Latent array $N \times D$	512×1024	96×384 (CIFAR-10), 128×512 (ModelNet40)
Cross-attend iterations T	8	4 (CIFAR-10), 2 (ModelNet40) — and T is itself an ablation axis
Positional encoding	2D Fourier, $K = 64$ bands/axis (258 dims)	**Identical**: 2D Fourier, $K = 64$ bands/axis, 258 dims + 3 RGB = 261
Weight sharing	On by default (shared across T)	Exposed as an ablation switch (on/off)
Model selection	—	**Validation carved from train** (45k/5k/10k); test touched once
Parameters	30–200M+	10.18M (CIFAR-10), 23.29M (ModelNet40), 18.87M (IO text)

Kept identical to the paper

Core mechanism preserved: cross-attention bottleneck → weight-shared latent transformer → mean-pooling (Perceiver) / output-query decoder (IO); GELU MLP, pre-norm LayerNorm, Fourier frequencies linearly spaced to Nyquist.

CIFAR-10: Experimental Setup

Goal: validate the Perceiver on raw pixels, with no convolutional assumption anywhere.

Hyperparameter	**Value**
Latent array ($N \times D$)	96×384
Cross-attn stages (T)	4, interleaved
Transformer blocks (L)	4 (shared)
Heads (cross / self)	1 / 8
Fourier bands / max freq	64 / 16
Dropout	0
Optimizer	LAMB
Learning rate	0.004
Scheduler	MultiStep 84/102/114
Batch size	64
Epochs	120, *no early stop*
Precision	bfloat16
Total parameters	**10 175 362**

Protocol — the part that makes the numbers usable

Train split: **45 000** images. Validation: **5 000**, carved out of the training set. Test: the official **10 000**, touched *once*, at the very end.

The reported checkpoint is the one with the best *validation* accuracy. Selecting the epoch on the same set you report on is **selection leakage** and inflates every number.

Data pipeline

- 32×32 RGB image $\rightarrow$ **1 024 pixels** as an unordered set
- Each pixel: 3 RGB + 2D Fourier PE = **261**

The Noise Band: How Much Does a Number Move on Its Own?

Three runs, identical in everything but the seed:

Run	Seed	Test acc.
`e01_baseline`	42	71.63%
`e31_baseline_seed1`	1	68.85%
`e32_baseline_seed2`	2	70.97%
Spread (max − min)		**2.78pp**

The rule for every slide that follows

Any difference smaller than **2.78 percentage points** is not an effect. It is seed variance.

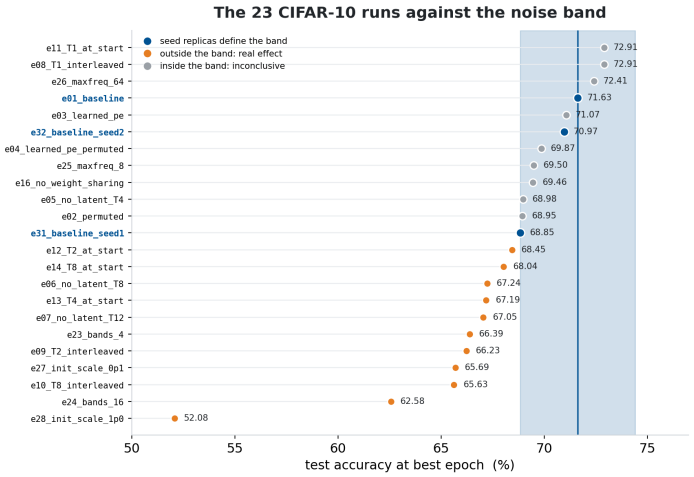
Why this slide exists

Two extra runs cost a few GPU-hours and change the status of all the others: without a measured band, every comparison is an opinion.

It also sets the honest ceiling of this study. On CIFAR-10 with 1 024 pixels and one GPU, most of the paper's ablations — measured on ImageNet with 50 176 pixels and 64 TPUs — simply do not have room to show up.

The band depends on the metric: 2.78pp on test accuracy, 2.94pp on best validation, 3.24pp on final validation. The conclusions do not change.

CIFAR-10: What Survives the Noise Band



23 runs, 11 conclusive

- Outside the band: latent init scale, cross-attend arrangement, Fourier bands, no latent transformer
- Inside: permutation, learned PE, weight sharing, max frequency

Source: `test_accuracy` at `selected_epoch`, read from `logs/<run>/results.json`.

CIFAR-10: Permutation Invariance — the Headline Claim

Comparison	Positional encoding	Natural → permuted	Δ
e01 → e02	Fourier	71.63% → 68.95%	−2.68
e03 → e04	learned	71.07% → 69.87%	−1.20
Paper (ImageNet): Fourier 78.0 → 78.0		learned 70.9 → 70.9	

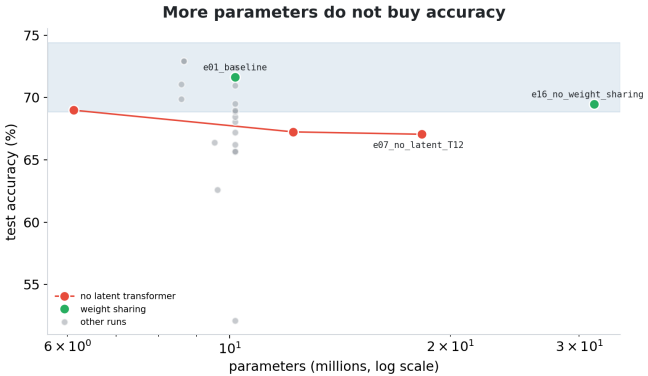
What we measured

Both drops fall **inside** the 2.78pp band: shuffling the pixels causes no measurable damage. Invariance confirmed — in the honest, weak form: *we did not measure a loss*, which is not the same as proving the loss is zero.

The subtle point

Invariance holds for **both** encodings. It comes from **attention** — a weighted sum has no order — not from the Fourier features. Fourier features buy *absolute* accuracy on ImageNet (78.0 vs 70.9), because learning 50 176 distinct positional embeddings is hard; they are not what makes the model permutation-proof.

CIFAR-10: Capacity Does Not Buy Accuracy



Weight sharing is free

- e01 71.63% with 10.2M params
- e16 69.46% with **31.5M** params
- 3.1× the weights, −2.17pp — inside the band

Where you put capacity matters

- Remove the latent transformer, add cross-attends instead:
- 6.1M → 68.98%
  12.2M → 67.24%
  18.3M → 67.05%
- **More parameters, less accuracy**

CIFAR-10: How Many Cross-Attends, and Where

T	interleaved	all at start	Δ (arrangement)
1	**72.91%**	**72.91%**	0.00
2	66.23%	68.45%	+2.22
4	**71.63%** (baseline)	67.19%	−4.44
8	65.63%	68.04%	+2.41

The one conclusive result

At $T = 4$, interleaving beats front-loading by **4.44pp** — outside the band. Re-reading the input *after* the latents have been refined lets each pass look for something different.

A reproducibility check, for free

At $T = 1$ the two arrangements produce the *same computational graph*, and indeed give bit-identical results: 72.91%, epoch 90, 8 654 662 params. Fixed seed ⇒ reproducible training.

Note the non-monotonicity along T: reading the input once is as good as reading it four times, with 15% fewer parameters.

CIFAR-10: The Hyperparameter That Actually Matters

Sweep	Value	Test acc.
Latent init σ	**0.02**	**71.63%**
	0.1	65.69%
	1.0	**52.08%**
Fourier bands	4	66.39%
	16	62.58%
	64	**71.63%**
Max frequency	8	69.50%
	16	**71.63%**
	64	72.41%

Latent initialisation: the only clean monotone effect

σ from 0.02 to 1.0 costs **19.55pp**. At $\sigma = 1.0$ the best epoch is **9 out of 120**: the model learns briefly, then degrades for the remaining 111 epochs.

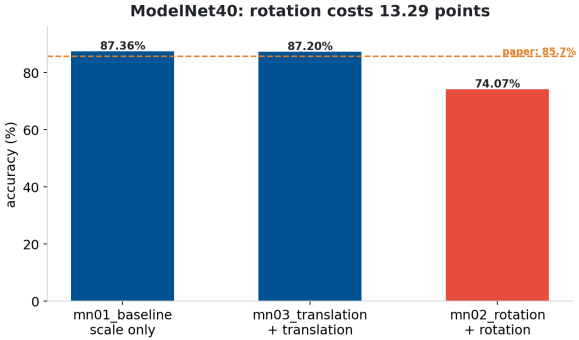
Two honest caveats

Bands is non-monotone (4 beats 16), and both low points are runs whose *final* validation collapsed (22.5% and 55.5%) — an optimisation problem, not a resolution one.

Max frequency sits entirely inside the noise band. The whole axis is inconclusive, and saying so is part of the result.

ModelNet40: Same Network, 3D Point Clouds

Setting	Value
Latent array	128×512
Cross-attn / blocks	2 / 6
Points per model	2 048
Input per point	387 dims
Fourier bands / max f	64 / 1120
Optimizer / lr	LAMB / 0.001
Batch (paper: 512)	**32**
Epochs	120
Parameters	23 288 928



Metric note: ModelNet40 has no separate validation split — the standard test set (2 468 objects) *is* the validation set, so the figure is `val_accuracy` at the best epoch.

Above the paper

87.36% vs the paper's **85.7%**, with a batch 16× smaller. Plausible: ModelNet40 is small and its objects arrive canonically aligned, so the large-batch advantage that dominates ImageNet counts for little here.

ModelNet40: Rotation Costs 13 Points — and Why

Augmentation	Accuracy	Δ
scale only	**87.36%**	—
+ translation	87.20%	−0.16
+ rotation	74.07%	−13.29

The largest effect in the whole project

And the one with the cleanest theoretical explanation.

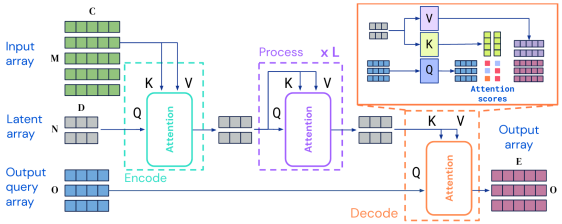
The Perceiver has no rotational inductive bias

3D Fourier PE encodes **absolute** position. Rotating an object changes every coordinate — and therefore every encoding — while the label stays the same. The network must learn from scratch an invariance the architecture does not grant, and that this task does not even need: ModelNet40 objects are already canonically aligned.

Translation is nearly harmless (−0.16) because it shifts all points equally: the *relative* structure survives.

Takeaway: a missing inductive bias cannot be patched with augmentation.

Perceiver IO: Structured Outputs



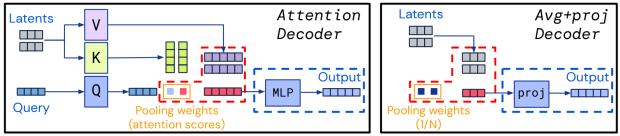
Extension over Perceiver (Jaegle et al., ICLR 2022):

- **Encode**: input $\rightarrow$ latent (same cross-attn encoder)
- **Process**: latent self-attention ($\times L$)
- **Decode**: output queries read latents

Why it matters:

- Original Perceiver: one pooled global output
- IO: task-specific output shape
- Same latent bottleneck, more general decoder

Perceiver IO Decoder with Output Queries



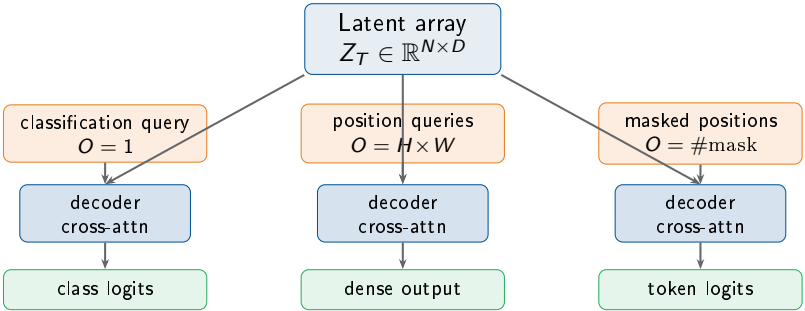
Decoder cross-attention (single layer):

$$Q_{\text{dec}} = \text{OutputQuery}\, W_Q, \quad K_{\text{dec}} = \text{Latent}\, W_K, \quad V_{\text{dec}} = \text{Latent}\, W_V$$

$$\text{Output} = \text{softmax}\left(\frac{Q_{\text{dec}} K_{\text{dec}}^\top}{\sqrt{d_k}}\right) V_{\text{dec}} \in \mathbb{R}^{O \times E}$$

- The number of output queries O is **task-dependent** and independent of N and M
- A single decoder cross-attention layer suffices (no self-attention among outputs)

Output Query Design per Task



Classification

One learned query produces one global prediction.

Dense outputs

Output shape is controlled by the query array.

Masked LM

Only masked positions need decoder queries.

Perceiver IO on CIFAR-10: Not Yet Measured

The question these runs will answer

Same encoder as `e01_baseline`, but the mean-pooling head is replaced by a **decoder with one output query**.

`io01_cifar` and its seed replica `io02_cifar_seed1` are in the registry; **neither has been run**.

What to expect, honestly

Very little. The query decoder earns its keep when the output is **structured** — a segmentation mask, an optical-flow field, 1 024 tokens. For a single label out of ten, querying the latents and averaging them should be equivalent.

The interesting outcome would be *no difference*: evidence that the decoder generalises the output shape at no cost in the simple case.

A number that used to be on this slide

Earlier versions of this deck reported **Perceiver IO 78.20% vs Perceiver 69.69%**, concluding the decoder was worth **+8.51pp**.

That comparison came from a first-generation run in which the **test set was also used as validation**, and in which three ablation flags were never read by the code.

It has been **removed, not updated**. Replacing it with an estimate would have meant inventing a result.

Paper reference (ImageNet): Perceiver IO **79.0%** with 2D Fourier features, **84.5%** with JFT pre-training — never using a convolution.

WikiText-103: Byte-Level Masked Language Model

Setting	Value
Latent array	128×512
Cross-attn / blocks	1 / 4
Heads (cross / self)	1 / 8
Sequence length	512 bytes
Vocabulary	$256 + 1$ mask
Masking rate	15%
Output queries	one per position
Optimizer / lr	LAMB / 0.001
Epochs / batch	10 / 32
Parameters	18 872 740
Masked-byte accuracy	**86.68%**
Chance level (1/256)	0.39%

The only real Perceiver IO result — and it is solid

$\sim$**222**$\times$ **better than chance**, reading raw UTF-8 with **no tokenizer at all**. Nearly nine masked bytes out of ten reconstructed.

Why bytes are affordable here

Cross-attention costs $O(M \cdot N)$ — *linear* in sequence length — instead of $O(M^2)$. Going from 512 tokens to 2 048 bytes costs BERT 16$\times$ in attention, and Perceiver IO only 4$\times$.

Read this number carefully

Masked-*byte* accuracy is not comparable to masked-*token* accuracy: the space is 256 rather than tens of thousands, and many bytes are fixed by orthographic context. It shows the

GLUE: Protocol Ready, Runs Pending

Run	Task	Ep.	Majority
`io_glue_cola`	acceptability	30	≈69%
`io_glue_sst2`	sentiment	10	≈50%
`io_glue_mrpc`	paraphrase	30	≈68%
`io_glue_stsb`	similarity (reg.)	30	—
`io_glue_qqp`	duplicate Q	3	≈63%
`io_glue_mnli`	NLI (3-way)	3	≈33%
`io_glue_qnli`	QA entailment	10	≈50%
`io_glue_rte`	entailment	30	≈50%
`*_scratch` ×2	**no pre-training**	—	—
`io_glue_multitask`	all 8, one query each	—	—

0 of 8 completed. The GLUE average comparable with Table 1 of the IO paper (81.0 byte-level / 81.1 BERT) **does not exist yet**.

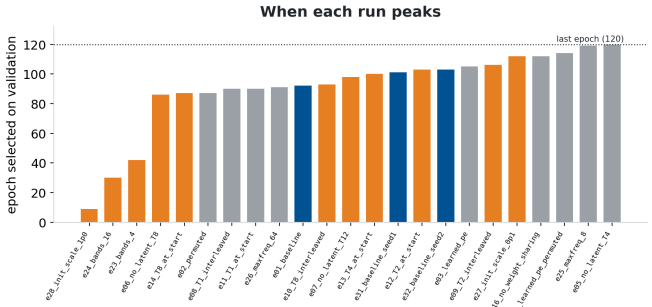
Fail-fast by design

Fine-tuning loads the encoder weights from the MLM checkpoint. If the checkpoint is missing the code **stops with an error** instead of silently training from scratch and calling it transfer.

The two control runs matter as much as the eight

A fine-tuned 61% means nothing on its own — maybe the task is solvable at 61% from random weights. The `scratch` controls are chosen at the extremes: **SST-2** has 67 000 training examples, **RTE** has 2 500. Transfer should matter little where data abounds and a lot where it is scarce.

Convergence: When Each Run Peaks



Why this matters

Because there is **no early stopping**, every run trains the full 120 epochs and the reported checkpoint is the best one on *validation*. The distribution of those epochs is itself a diagnostic.

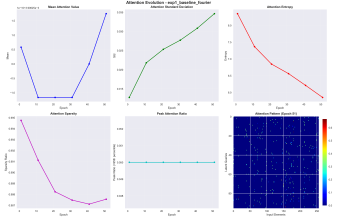
Three unstable runs

e23: best 67.74% @42, final **22.54%**
e24: best 62.66% @30, final **55.46%**
e28: best 53.36% @**9**, final 46.32%

Their reported numbers come from the validation-selected checkpoint — the correct methodology — but the instability has to be declared, not hidden.

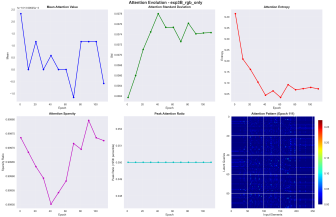
Attention Maps Analysis

Fourier PE (baseline, final maps)



Structured, spatially selective patterns (entropy ∼5.9). Latents specialize in different image regions.

RGB Only — no PE (final maps)



Diffuse, near-uniform patterns (entropy ∼9.1). No spatial selectivity — the model cannot localize information.

Interpretation

With Fourier PE, cross-attention learns spatially coherent patterns across training. Without PE, attention entropy remains high and patterns lack structure.

Comprehensive Results Summary

Modality	Model	Best Acc.	Reference	Experiments
Images (CIFAR-10)	Perceiver	72.91%	>90%†	**23** runs
Images (CIFAR-10)	Perceiver IO	not run	—	2 pending
Point Clouds (ModelNet40)	Perceiver	**87.36%**	85.7%	3 augmentations
Text (WikiText-103)	Perceiver IO	**86.68%** MLM	0.39% chance	1
NLU (GLUE, 8 tasks)	Perceiver IO	not run	81.0 (paper)	11 pending

† Strong ViT/CNN baselines on CIFAR-10; neither Perceiver paper benchmarks CIFAR-10, so this column is context, not a target.

What is measured

- ModelNet40 **beats** the paper: +1.66pp
- Best CIFAR-10 run: T=1, 72.91%, with 15% fewer parameters than the baseline
- Noise band **2.78pp**, measured on 3 seed replicas
- 11 of 23 CIFAR runs are conclusive

Scope of the campaign

- Declarative registry of **42 runs**, **27** completed
- 3 modalities: 2D images, 3D point clouds, raw bytes
- One command per run: `experiments.py -run <id>`
- **15 runs pending**, almost all on the Perceiver

Gap vs Original Papers

ModelNet40 (Point Clouds):

Source	Acc.
Original Paper	85.7%
This implementation	**87.36%**
Gap	**+1.66pp**

We are above the paper.
With a batch 16× smaller (32 vs 512). Plausible because ModelNet40 is small and canonically aligned, so the large-batch advantage counts for little.

CIFAR-10 (Images):

Source	Acc.
ViT/CNN baseline	>90%
Best Perceiver run	72.91%
Gap	**∼17pp**

Larger image gap.
Reference is strong ViT/CNN baselines — neither Perceiver paper benchmarks CIFAR-10. Scale gap: TPU cluster vs single GPU, 1 024 pixels vs 50 176.

Scale factors

The CIFAR-10 gap is primarily due to **batch size** (64 vs 1024), **latent count** (96 vs 256), and **compute time** limitations. The Perceiver architecture is particularly sensitive to these scaling factors on image data.

Hardware Limitations & Design Choices

Parameter	Original Paper (Google)	Project Setup
Hardware	512 TPU v3 cores	1 NVIDIA consumer GPU
Batch size	up to 1024	max 64–128
Latents (N)	256–512	96–128
Model size	30–100M+ params	3.35–10M params
Training time	days (distributed)	hours–days (single GPU)

Consequences:

- LAMB optimizer less effective with small batches
- Fewer latents limit representational capacity
- Reduced augmentation due to time constraints
- Fewer hyperparameter search iterations

Mitigation strategies:

- Weight sharing to reduce memory
- Gradient accumulation where possible
- Careful cosine annealing schedule
- Focus experiments on most informative ablations

Possible Improvements

Architecture:

- Increase latent count ($N = 256+$) with gradient checkpointing
- Add multi-scale cross-attention (coarse $\rightarrow$ fine)
- Explore relative positional encodings (RoPE, ALiBi)
- Deeper decoder for structured outputs

Training:

- Larger batch sizes with gradient accumulation
- Mixed precision (FP16/BF16) training
- Longer training schedules
- Advanced augmentation (CutMix, MixUp)

Data & Tasks:

- ImageNet pre-training for vision
- Subword tokenization for NLP
- Multi-modal joint training
- Audio modality experiments

Analysis:

- Layer-wise attention probing
- Latent space clustering visualization
- Systematic hyperparameter search
- Comparison with efficient Transformers (Linformer, Performer)

Empirical Summary

Conclusive (outside the 2.78pp band):

- **Generality**: one architecture, 87.36% on 3D point clouds — above the paper — and 86.68% on raw bytes
- **Where** capacity goes beats **how much**: 6.1M→12.2M→18.3M params gives 68.98→67.24→67.05%
- **Latent init scale** is the critical hyperparameter: 0.02→71.63%, 1.0→52.08%
- **Interleaving** cross-attends beats front-loading by 4.44pp
- **Rotation augmentation** costs 13.29pp on ModelNet40

Inside the band — "no measurable damage":

- Permutation invariance, with *both* positional encodings
- Weight sharing: 3.1× fewer parameters at no measurable cost
- Fourier vs learned PE at this scale

Not measured — runs still pending:

- Lower bound with no positional encoding
- Perceiver vs Perceiver IO on images
- The GLUE average (0 of 8 tasks)
- A convolutional baseline

Central methodological finding

Measuring the **noise band** — three replicas differing only by seed — cost two runs and changed the status of all the others. Without it, 9 of the 23 comparisons would have been reported as effects when they are variance.

Conclusions

Architecture properties:

- Single architecture for images, point clouds, and text
- Linear complexity in input size
- Memory efficiency via weight sharing ($\sim$61%)
- Zero CNN/RNN domain-specific assumptions
- Output queries enable flexible structured decoding

Remaining limitations:

- Requires large compute to match specialized models
- Heavily depends on positional encoding
- Outperformed by efficient CNNs on standard vision benchmarks
- LAMB optimizer introduces training instability at small scale
- Latent bottleneck limits fine-grained spatial reasoning

Closing statement

The experiments show that a **single general-purpose architecture** can be applied to images, point clouds, and text. At the tested scale, performance depends strongly on positional encoding, latent capacity, and available training compute.

Thank you!

Questions?

Francesco Gigli and Corso Vignoli

Code and models available upon request